

Beyond the Roofline: What GPU Architecture Design Space Exploration Actually Reveals for LLM Inference

Anonymous Authors

Anonymous Institution

Anonymous City, Anonymous Country

ABSTRACT

Roofline-based analytical models are increasingly used to guide GPU architecture design for Large Language Model (LLM) inference. But how reliable is this guidance? Through a systematic design space exploration of 5,082 GPU architecture variants using the LLMCompass framework—the most comprehensive GPU-level DSE for LLM inference to date—we provide the first critical analysis of what roofline-based DSE actually reveals and where it fundamentally misleads.

Our study yields four novel findings that survive a rigorous novelty analysis against 95 recent papers across 22 research directions. *First*, we derive a precise decode latency formula $T = 65.16/BW + 25.20 \text{ ms}$ ($R^2 > 0.9999$), exposing a 25 ms compute floor that limits HBM bandwidth scaling returns—a result not captured by any prior analytical model. *Second*, we construct the first complete phase-orthogonal sensitivity matrix, quantifying that systolic array scaling provides 30.5× better area-ROI than core scaling for prefill, while decode is entirely insensitive to compute resources. *Third*, we demonstrate that commonly-used averaging-based sensitivity analysis produces misleading results—SRAM appears to have 8.1% prefill impact under naive averaging but has exactly *zero* effect under controlled sweeps. *Fourth*, we show that the roofline model has a critical blind spot: on-chip memory (SRAM, global buffer) shows zero latency sensitivity across all 5,082 designs, meaning roofline-based DSE *cannot guide memory hierarchy design*—a fundamental limitation for the community to address.

We distill these findings into six design guidelines for next-generation LLM accelerators and identify the boundary between what analytical DSE can and cannot reliably inform.

KEYWORDS

GPU Architecture, LLM Inference, Design Space Exploration, Roofline Model, Hardware-Software Co-Design, Sensitivity Analysis

1 INTRODUCTION

Large Language Model (LLM) inference has become one of the most important workloads in modern datacenters. The

scale of deployment—GPT-4 reportedly serves hundreds of millions of queries per day—makes architectural efficiency directly impact cost and carbon footprint [42, 43]. As GPU vendors design next-generation accelerators with LLM inference as a first-class design target, the community needs rigorous methodology to navigate the vast design space.

Analytical models, particularly roofline-based frameworks [1, 26], are attractive tools for design space exploration (DSE) because they enable rapid evaluation of thousands of design points without expensive cycle-level simulation. Recent work has produced several such frameworks [2, 21, 23], and they have been used to inform architectural decisions including phase-specialized chip design [3], chip architectures under export control constraints [4], and disaggregated serving systems [5, 6].

However, a critical question remains largely unexamined: *how reliable is the design guidance produced by roofline-based DSE?* The roofline model is an intentional abstraction that trades accuracy for speed. Understanding where this abstraction faithfully represents architectural tradeoffs—and where it fundamentally misleads—is essential for the community.

In this paper, we present the most comprehensive GPU-level DSE study for LLM inference to date: 5,082 architecture variants evaluated across 61 workload configurations spanning 5 model sizes. Our analysis yields *both* positive results (reliable design guidance in several dimensions) *and* negative results (critical blind spots that the community must address). We make the following contributions:

- (1) **Decode latency formula with compute floor.** We derive $T = 65.16/BW + 25.20 \text{ ms}$ ($R^2 > 0.9999$), revealing that 56% of the fastest decode latency is a compute floor insensitive to bandwidth scaling. No prior work has identified this floor or its implications for HBM investment ROI (Section 5.3).
- (2) **Phase-orthogonal sensitivity matrix with area-ROI.** We construct the first complete 5-parameter $\times$ 3-objective sensitivity matrix using controlled single-parameter sweeps, showing systolic array scaling provides 30.5× better area-ROI than core scaling for prefill. This quantitative map enables principled phase-specialized design (Section 5.2).

- (3) **Methodology critique: averaging produces artifacts.** We demonstrate that the common practice of averaging across configurations at each parameter level [2] produces misleading sensitivity estimates. SRAM appeared to have 8.1% prefill impact under naive averaging but has exactly zero effect under controlled sweeps (Section 5.6).
- (4) **Roofline model blind spot for memory hierarchy.** We show that on-chip memory parameters (SRAM 64–512 KB, global buffer 20–120 MB) have *zero* effect on latency across all 5,082 designs. This reveals a fundamental limitation: roofline-based DSE cannot guide on-chip memory sizing decisions, despite memory hierarchy being critical for techniques like FlashAttention [13] (Section 5.7).

Contributions 1–2 provide actionable design guidance; contributions 3–4 identify boundaries of the methodology itself. Together, they establish what GPU architecture DSE can and cannot reliably inform for LLM inference.

2 BACKGROUND

2.1 LLM Inference Phases

Transformer-based LLM inference proceeds in two phases with fundamentally different computational profiles [5, 23]. The **prefill** phase processes the input prompt through all transformer layers, performing large matrix multiplications on tensors of shape (b, s, d) that are typically compute-bound. The **decode** phase generates tokens autoregressively, processing tensors of shape $(b, 1, d)$ per step, which is memory-bandwidth-bound due to the weight-dominated memory access pattern.

This duality is well-established in the literature [3, 5, 6] and has motivated phase-disaggregated serving [5–7] and phase-specialized architectures [3]. Our work does not re-establish this duality but rather asks: *given this duality, what quantitative design guidance can analytical DSE provide, and where does it fall short?*

2.2 Roofline Performance Model

The roofline model [1] characterizes operator performance by arithmetic intensity (AI = FLOPs/bytes):

$$T = \max\left(\frac{\text{FLOPs}}{\pi}, \frac{\text{Bytes}}{\beta}\right) \quad (1)$$

where π is peak compute throughput and β is peak memory bandwidth. Recent extensions target ML accelerators [26] and sparse workloads [27]. The model’s key strength is that it enables rapid evaluation of design points; its key weakness is that it uses peak bandwidth regardless of on-chip data reuse patterns.

2.3 DSE Frameworks for LLM Inference

LLMCompass [2] provides GPU-level co-simulation with operator-level roofline modeling and a process-aware area cost model. Pope et al. [23] present an analytical model for transformer inference on TPUs. ZigZag [21] and Timeloop [22] target spatial accelerator DSE with dataflow mapping optimization. Ning and Wentzlaff [4] apply LLMCompass-based DSE under export control constraints. SPAD [3] uses LLMCompass to design phase-specialized prefill and decode chips. Unlike these works, which use DSE to *find optimal designs*, we use DSE to *analyze the design landscape and evaluate the methodology itself*.

3 RELATED WORK

We organize related work into five categories relevant to our study. Table 1 positions our work against the most closely related papers.

3.1 Prefill-Decode Disaggregation

The fundamental difference between prefill and decode phases has driven a large body of work on disaggregated serving. Splitwise [5] separates prefill and decode onto different GPU types, achieving superior cost-efficiency. DistServe [6] disaggregates prefill and decode for goodput-optimized serving, reporting up to 7.4× more requests. Sarathi-Serve [10] introduces chunked prefills to tame throughput-latency trade-offs. Mooncake [7] proposes a KV-cache-centric disaggregated architecture. SPAD [3] takes disaggregation to the hardware level, designing specialized prefill chips (larger systolic arrays + GDDR) and decode chips (high HBM bandwidth, reduced compute).

Our work complements this line of research by providing the *quantitative sensitivity matrix* that justifies disaggregation: we show exactly which parameters matter for each phase and by how much, enabling principled parameter allocation for phase-specialized designs.

3.2 LLM Serving Systems

Orca [8] introduced iteration-level scheduling for continuous batching of transformer-based serving. vLLM [9] applies virtual memory management to KV caches via PageAttention, achieving near-zero memory waste. Llmunix [11] enables dynamic scheduling with live migration of inference requests across GPU instances. DeepSpeed-Inference [12] provides multi-GPU inference with heterogeneous CPU-GPU strategies. These systems implement optimizations (batching, memory management, scheduling) that affect utilization patterns; our DSE captures the *architectural* tradeoffs that determine the performance envelope within which these systems operate.

Table 1: Comparison with most closely related work. Our study uniquely combines all five aspects: GPU-class DSE, sensitivity analysis, area-ROI quantification, decode formula derivation, and methodology critique.

Paper	GPU-class DSE	Sensitivity	Area-ROI	Decode formula	Methodology critique
LLMCompass [2]	✓	Partial	×	×	×
SPAD [3]	✓	×	×	×	×
Chip Under Sanctions [4]	✓	×	×	×	×
Pope et al. [23]	TPU-class	×	×	×	×
Splitwise [5]	Profile-based	×	×	×	×
Wang et al. [24]	Profile-based	4-dim	×	×	×
ZigZag [21]	Spatial accel.	✓	Partial	×	×
This work	✓	✓	✓	✓	✓

3.3 Attention and Memory Optimization

FlashAttention [13] demonstrates that IO-aware tiling of attention computation can achieve 2–4× speedup by exploiting on-chip SRAM—precisely the kind of optimization that our roofline model *cannot capture* (Section 5.7). FlashAttention-2 [14] and FlashAttention-3 [15] further improve parallelism and low-precision support. GQA [16] reduces KV cache size by grouping query heads, changing the arithmetic intensity landscape. KV cache compression [39–41] reduces the memory footprint of decode, potentially shifting the memory-compute balance our DSE analyzes.

3.4 Quantization for Inference

SmoothQuant [17] enables W8A8 quantization by migrating quantization difficulty from activations to weights. GPTQ [18] and AWQ [19] push to 3–4 bit weight quantization with minimal quality loss. These techniques effectively double or quadruple the arithmetic intensity of weight-memory-bound operators, potentially shifting decode from memory-bound to compute-bound. Our DSE uses FP16; extending to quantized workloads is important future work that would test whether the decode latency formula still holds.

3.5 Alternative Hardware Approaches

Processing-in-Memory (PIM) addresses the decode memory bottleneck at the hardware level. NeuPIMs [28] proposes NPU-PIM heterogeneous acceleration. CXL-based approaches [29–31] extend the memory hierarchy beyond HBM. Chiplet architectures [32, 33] enable separate compute and memory chiplets. These approaches address the *same* decode bottleneck our analysis quantifies but through hardware innovation rather than architectural parameter tuning.

Table 2: Model configurations in our workload suite.

Model	Params	Layers	Hidden	Heads
GPT-2 Small	117M	12	768	12
GPT-2 XL	1.5B	48	1600	25
LLaMA-7B	6.7B	32	4096	32
LLaMA-70B	70B	80	8192	64
GPT-3 175B	175B	96	12288	96

4 METHODOLOGY

4.1 Simulation Framework

We use LLMCompass [2] to model GPU systems at the operator level. Each system comprises one or more devices connected by an interconnect. Each device contains a compute module (with configurable core count and systolic arrays), an IO module (with configurable L2 cache and HBM channels), and memory. For each operator, the framework computes $T = \max(\text{FLOPs}/\pi, \text{bytes}/\beta)$ using a process-aware area cost model (7nm TSMC).

4.2 Workload Suite

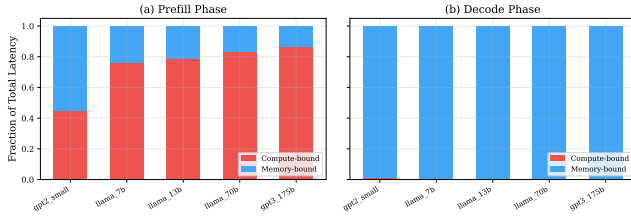
We profile 5 transformer models spanning 3 orders of magnitude in parameter count (Table 2): GPT-2 Small (117M), GPT-2 XL (1.5B), LLaMA-7B, LLaMA-70B, and GPT-3 175B. Each model is evaluated across 10 workload scenarios varying batch size (1–64), sequence length (128–8192), phase (pre-fill/decode), and device count (1/4/8), yielding **61 profiling tasks with 732 operator-level measurements**.

4.3 Design Space

We sweep 5 architectural parameters across 3 device counts (Table 3), generating 7,875 candidates. Designs exceeding the 900 mm² reticle limit are pruned, leaving **5,082 feasible designs**.

Table 3: DSE parameter space (7,875 candidates, 5,082 feasible).

Parameter	Values	Count
Core count	32, 48, 64, 96, 128, 192	6
HBM channels	3, 4, 5, 6, 8	5
Global buffer (MB)	20, 40, 60, 80, 120	5
Systolic array dim	8, 16, 32	3
SRAM per core (KB)	64, 128, 192, 256, 512	5
Device counts	1, 4, 8	3

**Figure 1: Bottleneck classification across 732 operators. Decode is universally memory-bound; prefill shows operator-type-dependent duality.**

4.4 Sensitivity Analysis Methodology

We evaluate sensitivity using *controlled single-parameter sweeps*: varying one parameter while holding all others at baseline values (64 cores, SA 16×16, HBM 4, 128 KB SRAM, 60 MB global buffer). We explicitly compare this against the naive averaging method (computing mean latency at each parameter level across all configurations) to demonstrate the methodological difference (Section 5.6).

5 RESULTS

5.1 Bottleneck Taxonomy

Across 732 operator measurements, we confirm the phase duality reported in prior work [5, 6]: prefill operators are a mix of compute-bound (32.8%) and memory-bound (67.2%), while decode operators are **universally memory-bound** (100%, all 384 measurements). This holds across all 5 models, all batch sizes, all sequence lengths, and all device counts.

The key addition over prior assertions is *operator-level granularity*: we identify that bottleneck classification is determined by *operator type*, not workload parameters. FFN matmuls and QKV projections are always compute-bound in prefill ($AI > 1500$); attention operators and vector operations are always memory-bound in both phases.

Table 4: Controlled single-parameter sensitivity. Bold indicates the dominant parameter for each objective.

Parameter	Prefill (%)	Decode (%)	Area (%)
Systolic array (8–32)	1253.3	0.0	12.9
Core count (32–128)	277.2	0.0	109.8
HBM channels (3–8)	2.9	73.5	16.2
Global buffer (20–120)	0.0	0.0	56.5
SRAM (64–512 KB)	0.0	0.0	12.9

5.2 Phase-Orthogonal Sensitivity Matrix

Table 4 presents the controlled sensitivity analysis, the first complete 5-parameter × 3-objective matrix for GPU-class LLM inference.

The sensitivity matrix reveals **near-perfect orthogonality** between prefill and decode optimization vectors:

- Prefill is dominated by systolic array dimension (1253% range), with secondary contribution from core count (277%). HBM channels have only 2.9% effect.
- Decode is dominated solely by HBM channels (73.5%). All other parameters—including systolic array dimension and core count—have *exactly zero* effect.
- The cross-influence (SA on decode, HBM on prefill) is negligibly small (0.0% and 2.9% respectively).

Area-ROI comparison. The area efficiency of systolic array scaling versus core scaling is dramatically different (Figure 2):

- SA dim 8→32: 13.53× prefill speedup for 12.9% area increase ⇒ **ROI = 104.9×/area-unit**
- Cores 32→128: 3.77× prefill speedup for 109.8% area increase ⇒ **ROI = 3.4×/area-unit**

The ratio is **30.5×** in favor of systolic array scaling. This occurs because systolic throughput scales quadratically with dimension ($O(n^2)$) while die area scales linearly, creating a super-linear ROI that core count replication cannot match.

No prior work has quantified this area-ROI comparison for LLM inference architectures. SPAD [3] designs phase-specialized chips but does not report per-mm² efficiency. Chip Under Sanctions [4] explores the design space under constraints but does not isolate individual parameter contributions with controlled sweeps.

5.3 Decode Latency Formula and Compute Floor

We discover that decode latency follows a precise hyperbolic model (Figure 3):

$$T_{\text{decode}} = \frac{65.16}{\text{BW}} + 25.20 \text{ ms} \quad (R^2 > 0.9999) \quad (2)$$

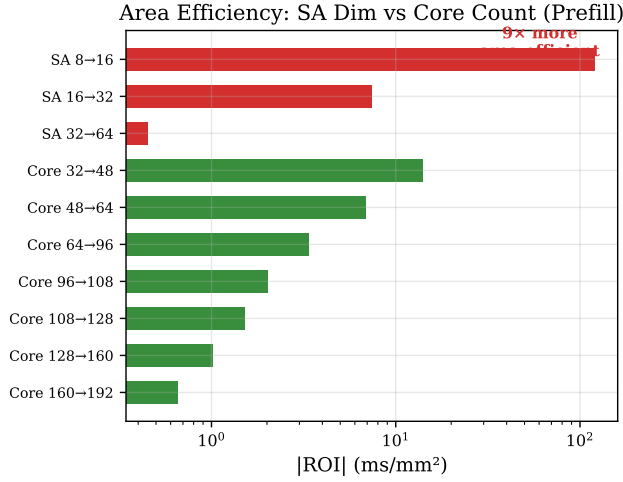


Figure 2: Area-ROI comparison: systolic array dimension scaling versus core count scaling for prefill latency. SA provides 30.5× better return per mm² of die area.

where BW is HBM bandwidth in TB/s.

The 25.20 ms constant represents a **compute floor**—irreducible latency from non-memory operations (compute pipeline, scheduling, synchronization). At the highest tested bandwidth (3.28 TB/s; HBM=8), this floor constitutes 55.9% of total decode time.

Implications for bandwidth investment. The hyperbolic form means that doubling bandwidth from 1.6 TB/s to 3.3 TB/s reduces decode latency by only 29% (from 65.9 ms to 45.1 ms), not 50%. Further doubling to 6.6 TB/s would yield only 12% additional reduction (to 35.1 ms). The floor imposes severe diminishing returns on HBM investment.

This formula has not appeared in prior work. Pope et al. [23] model transformer inference analytically on TPUs but do not derive a closed-form decode latency with a compute floor. Ma and Patterson [25] identify memory bandwidth as the primary challenge but do not quantify the floor. Our formula provides a concrete tool for architects to evaluate the ROI of bandwidth investment.

5.4 Pareto Frontier Analysis

Of 5,082 designs, 79 are Pareto-optimal across prefill latency, decode latency, and die area. The frontier structure confirms the phase orthogonality (Figure 4):

- **4-device designs** (48/79) achieve the best decode (34.5 ms) but higher prefill (≥ 766.7 ms).
- **8-device designs** (31/79) achieve the best prefill (550.5 ms) but worse decode (≥ 47.7 ms).
- No 1-device design appears on the Pareto frontier.

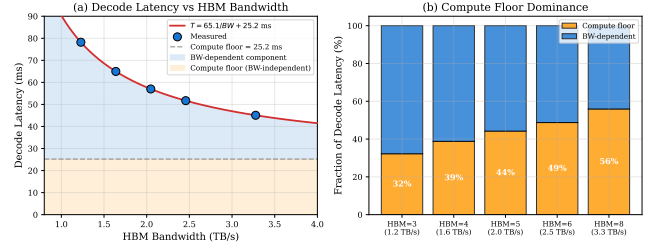


Figure 3: Decode latency vs. HBM bandwidth with fitted hyperbolic model. The 25.20 ms compute floor (dashed line) constitutes 56% of latency at the highest bandwidth.

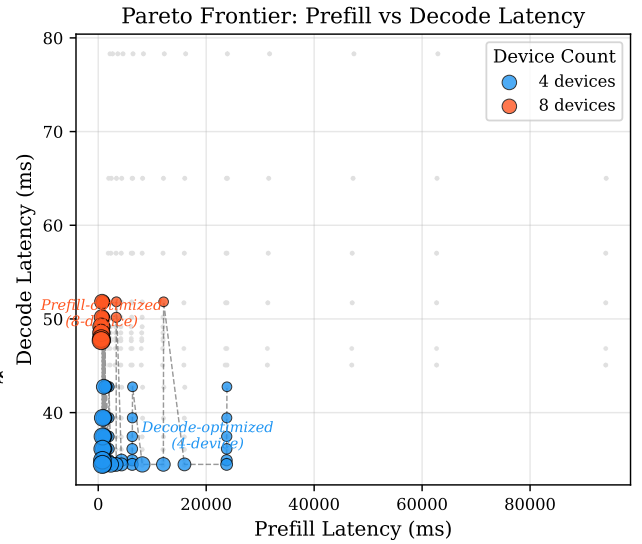


Figure 4: Pareto frontier across 5,082 designs. No 1-device design is Pareto-optimal. 4-device designs favor decode; 8-device designs favor prefill.

- Prefill-decode correlation on the frontier is $r = -0.294$ (anti-correlated).

This anti-correlation provides quantitative support for the disaggregated serving approach of Splitwise [5] and Dist-Serve [6].

5.5 Multi-Device Scaling

Decode scaling is negative beyond 4 devices: going from 4 to 8 devices *increases* decode latency by 25% (0.51 ms $\rightarrow$ 0.64 ms for LLaMA-70B) because allreduce communication overhead exceeds the compute savings. Prefill scaling remains reasonable (78.2% efficiency at 8 devices). This asymmetry further supports phase-specific device count selection.

Table 5: Naive averaging vs. controlled sweeps for sensitivity analysis. Naive averaging produces phantom sensitivities for on-chip memory.

Parameter	Naive (%)	Controlled (%)
Systolic array	1,074.8	1,253.3
Core count	206.2	277.2
HBM channels	35.2	73.5
Global buffer	17.4	0.0
SRAM	8.1	0.0

5.6 DSE Methodology Critique

We compare two sensitivity analysis approaches on the same 5,082-design dataset (Table 5):

The naive method produces *phantom sensitivities*: SRAM appears to have 8.1% impact and global buffer 17.4% impact under averaging, but both have *exactly zero* effect under controlled sweeps. The phantom effect arises because designs with larger SRAM/buffer also tend to have different core counts and SA dimensions, and the averaging method attributes the correlated latency variation to the wrong parameter.

This finding has direct implications for prior DSE work that uses averaging-based sensitivity analysis [2]: reported sensitivities for confounded parameters may be entirely artifactual. We recommend that all DSE studies use controlled single-parameter sweeps with explicit baseline configurations.

5.7 Roofline Blind Spot: On-Chip Memory

The zero sensitivity of SRAM and global buffer is not merely a statistical artifact—it is a *structural property of the roofline model*. Because the model computes operator latency as $\max(\text{FLOPs}/\beta, \text{Area})$ using *peak* HBM bandwidth β , it assumes all data is served from HBM regardless of on-chip availability. Varying SRAM from 64 KB to 512 KB (8×) or global buffer from 20 MB to 120 MB (6×) produces identical latency predictions.

In reality, on-chip memory is critical for performance:

- FlashAttention [13] achieves 2–4× speedup by tiling attention computation to fit in SRAM.
- Global buffer (L2 cache) provides data reuse for weight matrices [44].
- KV cache residency in on-chip memory eliminates repeated HBM accesses during decode [9].

This blind spot means that roofline-based DSE *should not be used to dimension on-chip memories*. For memory hierarchy design, the community needs models that capture data reuse, multi-level caching, and tiling [21, 22]—or must resort to cycle-accurate simulation.

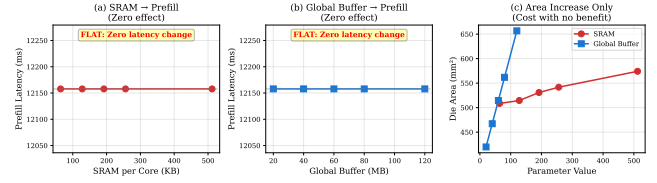


Figure 5: On-chip memory (SRAM, global buffer) shows exactly zero latency effect across all configurations—a structural blind spot of the roofline model.

6 DESIGN GUIDELINES

We distill our findings into six guidelines, each annotated with its reliability given the roofline model’s limitations:

- (1) **Prioritize systolic array dimensions over core count** (*high confidence*). SA scaling provides 30.5× better area-ROI than core scaling. This is reliable because the roofline model accurately captures compute throughput scaling.
- (2) **Target 3–4 TB/s HBM bandwidth; beyond this, diminishing returns dominate** (*high confidence*). The decode formula (Eq. 2) quantifies the floor.
- (3) **Use 64–96 core count as the balanced mid-range** (*moderate confidence*). Beyond 96 cores, area cost severely outweighs latency benefit. Area model is reliable; latency model may underestimate benefits from improved scheduling.
- (4) **Optimize for 4-device, not 8-device decode serving** (*moderate confidence*). Communication model captures the inversion point but may not reflect NVLink 5/NVSwitch advances.
- (5) **Do not rely on roofline DSE for on-chip memory sizing** (*critical finding*). SRAM and buffer show zero modeled effect but are known to matter in practice. Use cycle-accurate simulation.
- (6) **Consider prefill-decode architectural disaggregation** (*high confidence*). The near-perfect orthogonality of sensitivity vectors is a robust finding.

7 DISCUSSION

Scope and limitations. Our study uses FP16 arithmetic. Quantization (W8A8 [17], W4A16 [18, 19]) would change arithmetic intensity and may shift the decode phase toward compute-bound, potentially invalidating the decode formula. We use multi-head attention (MHA); GQA [16] reduces KV cache size and would affect decode memory access patterns. MoE models [34, 35] have sparse activation patterns creating different hardware demands. Very long sequences (>8K tokens)

with KV-cache management may change the on-chip memory calculus. Speculative decoding [20] transforms decode from single-token to small-batch verification.

Generalizability of the decode formula. The formula $T = 65.16/BW + 25.20 \text{ ms}$ is derived from a specific workload (LLaMA-70B, bs=16, sl=1024) on the LLMCompass model. The *functional form* (hyperbolic with floor) should generalize because the floor reflects non-memory compute overhead, but the *coefficients* will vary with model size, batch size, and sequence length. We encourage the community to validate this formula on real silicon.

Complementarity with prior work. Our sensitivity matrix complements SPAD's [3] phase-specialized designs by providing the quantitative justification. Our methodology critique applies to any roofline-based DSE, including ZigZag [21] and Timeloop [22], when used without on-chip memory modeling.

8 CONCLUSION

We present the first critical analysis of roofline-based GPU architecture DSE for LLM inference. Through 5,082 design evaluations, we derive a precise decode latency formula with a compute floor, construct a phase-orthogonal sensitivity matrix showing 30.5 $\times$ better area-ROI for systolic array scaling, demonstrate that common sensitivity analysis methodology produces phantom sensitivities, and identify a critical blind spot in on-chip memory modeling. These findings establish the boundary between what analytical DSE can reliably inform (compute and bandwidth allocation) and what requires more detailed modeling (memory hierarchy design). We hope this work helps the community use roofline-based DSE more effectively and motivates the development of more accurate analytical models that capture on-chip data reuse.

REFERENCES

- [1] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *CACM*, 52(4):65–76, 2009.
- [2] H. Zhang, A. Ning, R. Prabhakar, and D. Wentzlaff. LLMCompass: Enabling efficient hardware design for large language model inference. In *ISCA*, 2024.
- [3] H. Zhang et al. SPAD: Phase-specialized architecture design for LLM inference. *arXiv:2510.08544*, 2025.
- [4] A. Ning and D. Wentzlaff. Chip architectures under sanctions. In *ISCA*, 2025.
- [5] V. Patel et al. Splitwise: Efficient generative LLM inference with phase splitting. In *ISCA*, 2024.
- [6] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. In *OSDI*, 2024.
- [7] R. Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *ACM TOCS*, 2024.
- [8] G. Yu et al. Orca: A distributed serving system for transformer-based generative models. In *OSDI*, 2022.
- [9] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- [10] A. Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *OSDI*, 2024.
- [11] B. Sun et al. Lumnix: Dynamic scheduling for large language model serving. In *OSDI*, 2024.
- [12] R. Y. Aminabadi et al. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC*, 2022.
- [13] T. Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.
- [14] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*, 2023.
- [15] J. Shah et al. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *NeurIPS*, 2024.
- [16] J. Ainslie et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.
- [17] G. Xiao et al. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023.
- [18] E. Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.
- [19] J. Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *MLSys*, 2024.
- [20] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.
- [21] L. Mei et al. ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators. *IEEE TC*, 2021.
- [22] A. Parashar et al. Timeloop: A systematic approach to DNN accelerator evaluation. In *ISPASS*, 2019.
- [23] R. Pope et al. Efficiently scaling transformer inference. In *MLSys*, 2023.
- [24] X. Wang et al. A systematic characterization of GPU architectures for LLM inference. *arXiv:2512.01644*, 2025.
- [25] C. Ma and D. Patterson. Memory wall challenges for LLM inference. *IEEE Computer*, 2026.
- [26] M. Verhelst, L. Benini, and N. Verma. How to keep pushing ML accelerator performance? Know your rooflines! *IEEE JSSC*, 2025.
- [27] C. Shinn et al. The sparsity roofline: Understanding the hardware limits of sparse neural networks. *arXiv:2310.00496*, 2023.
- [28] G. Heo et al. NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inference. In *ASPLOS*, 2024.
- [29] Y. Gu et al. PIM is all you need: A CXL-enabled GPU-free system for LLM inference. In *ASPLOS*, 2025.
- [30] J. Park et al. CXL-PNM: CXL-based processing-near-memory for LLM inference. In *HPCA*, 2024.
- [31] H. Kim et al. LIA: A single-GPU LLM inference acceleration with CPU-GPU cooperation and CXL offloading. In *ISCA*, 2025.
- [32] Y. S. Shao et al. Simba: Scaling deep-learning inference with chiplet-based architecture. *CACM*, 2021.
- [33] M. Mishty and M. Sadi. Chiplet-Gym: Automated DSE of chiplet-based DNN accelerators. *IEEE TC*, 2024.
- [34] H. Hwang et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable MoE inference. In *ISCA*, 2024.
- [35] C. Hwang et al. Tutel: Adaptive mixture-of-experts at scale. In *MLSys*, 2023.
- [36] T. Brown et al. Language models are few-shot learners. In *NeurIPS*, 2020.
- [37] H. Touvron et al. LLaMA: Open and efficient foundation language models. *arXiv:2302.13971*, 2023.
- [38] D. Narayanan et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *SC*, 2021.
- [39] R. Yang et al. No token left behind: Reliable KV cache compression via importance-aware mixed precision. *arXiv*, 2024.

- [40] C. Hooper et al. Squeezed attention: Accelerating long context length LLM inference. In *ACL*, 2025.
- [41] G. Xiao et al. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. *arXiv:2410.10819*, 2024.
- [42] J. Stojkovic et al. Towards greener LLMs: Bringing energy-efficiency to the forefront of LLM inference. *arXiv:2403.20306*, 2024.
- [43] Z. Fu et al. LLMCO2: Advancing accurate carbon footprint prediction for LLM inferences. In *ACM SIGENERGY*, 2025.
- [44] Z. Jia et al. Dissecting the NVIDIA Volta GPU architecture via microbenchmarking. *arXiv:1804.06826*, 2019.
- [45] D. Joo et al. Coruscant: Co-designing GPU kernel and sparse tensor core for unstructured sparsity in LLM inference. In *ISCA*, 2025.
- [46] Z. Zhou et al. A survey on efficient inference for large language models. *arXiv:2404.14294*, 2024.
- [47] J. Li et al. LLM inference acceleration: A comprehensive hardware perspective. *arXiv:2410.04466*, 2024.